

Lab 03 - Divide & Conquer (Part 1)

Instruction:

- This lab is divided into two parts. The objectives of this part (Part 1) are to:
 - Define the binary search algorithm.
 - Generate its recurrence relation and derive its running time using the Master Theorem.

- The binary search algorithm is:

`BinarySearch(A, i, j, obj)`

01. if $i \leq j$, then

01. $m \leftarrow \lfloor (i+j)/2 \rfloor$

02. if $A[m] = obj$, then return m .

03. if $A[m] < obj$, then return the result of calling `BinarySearch()` on the interval $[m+1, j]$.

04. else, return the result of calling `BinarySearch()` on the interval $[i, m-1]$.

02. return -1 .

Note that this algorithm works only if A is sorted in ascending order.

- Submit one `.h` file and one `.cpp` source file.
- Your files may include only the following libraries: `iostream`, `string`, `vector`, `ctime`, and `cstdlib`. No others are permitted.
- Submit your files to your GitHub repository, inside a directory named `Lab03`.
- Cheating of any kind is prohibited and will not be tolerated.
- **Violating or failing to follow any of these rules will result in an automatic zero (0) for the lab.**

At the very beginning of your submission, add a comment containing your name and the date, to confirm you have read and understood the instructions above.

Grading

Task	Maximum Points	Points Earned
1	0.5	
2	0.5	
3	0.5	
4	0.5	
Total	2.0	

Task 1

In the header file, within the namespace *aal*, define the function:

```
template <class T>
int BinarySearch(const vector<T>& v, int s, int e, const T& target)
```

that returns the index of *target* in *v* using the binary search algorithm shown above, or -1 if *target* is not found.

Task 2

Directly below your definition of `BinarySearch()`, add a comment block that does the following:

1. States the recurrence relation of `BinarySearch()`, and explains the values of each of its components (a , b , and $f(n)$).
2. Derives its running time using the Master Theorem.

Show all of your work.

Task 3

In the header file, within the namespace *aal*, define the function:

```
template <class T>
void Shuffle(vector<T>& v)
```

that randomly shuffles the elements of *v*.

Task 4

In the `main` function, verify that your `BinarySearch()` function is correct by completing the following steps:

1. Initialize two integer vector objects to the consecutive integers from 1 to 40.
2. Shuffle the contents of one of the vector objects by invoking `Shuffle()` with that object as its argument.
3. Repeat the following a fixed number of times:
 - a) Prompt the user for a value to search for.
 - b) Invoke `BinarySearch()` on each vector object over the full index range $[0, 39]$, and display both results.